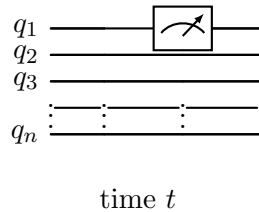


1 Week 9.

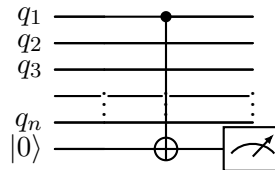
1. [Principle of Deferred Measurement.]

The point of this problem is to show that if one has a quantum circuit with (partial) measurement gates in the middle, one can (without much loss in efficiency) replace it with an equivalent quantum one in which all the measurement gates are at the end. This is nice, because a very useful simplifying assumption in quantum computation is that measurement gates only occur at the end of the computation.

So suppose we have some n -qubit quantum circuit, and we look at the first intermediate measurement gate that is applied; say (without loss of generality) it is applied to the 1st qubit, at time step t . Let $|\psi\rangle$ denote the quantum state just prior to time t . Now when the measurement gate is applied, two things happen: First, one classical bit of information — call it b — appears on the measurement gate's readout. Second, the state collapses according to the usual rules.



- (a) Suppose we do the following: First, we introduce a new $(n+1)$ st qubit, initialized in the state $|0\rangle$. Second, we replace the measurement gate on qubit q_1 at time t with a CNOT gate whose control qubit is q_1 and whose target qubit is q_{n+1} . Finally, we immediately apply a measurement gate to the $(n+1)$ st qubit, and treat its readout as " b ". Assume we then henceforth ignore the (collapsed) $(n+1)$ st qubit. Show that this gives an exact simulation of the original circuit's operation. (Hint: you may want to somehow write $|\psi\rangle = \alpha|0\rangle \otimes |\psi_0\rangle + \beta|1\rangle \otimes |\psi_1\rangle$.)



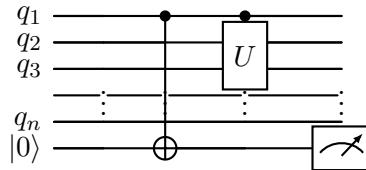
Remark: Note that operations that are applied to disjoint sets of qubits commute ($(U \otimes I) \cdot (I \otimes V) = (I \otimes V) \cdot (U \otimes I) = U \otimes V$, and similarly for the commuting of partial measurements). Thus, if we're never going to do anything with that $(n+1)$ st qubit again, we can imagine that instead of measuring it immediately (just after time t), we instead delay its measurement to the very end of the computation. In this way, we've effectively deferred the first intermediate measurement of the quantum circuit to the end. By repeating this for all intermediate measurement gates, we can

always move all measurement gates to the end (at the cost of adding one extra qubit and CNOT per deferral).

- (b) Let's look back at the original circuit and see “what was done” with the first qubit after it was measured. In some cases, that measurement gate was there because we genuinely wanted to know the 1 bit of classical information, b . In other cases, we don't care about b 's value per se; rather, we just want to do different subsequent quantum operations to the other qubits, depending on whether $b = 0$ or $b = 1$. (The Quantum Teleportation scenario is a bit like this.) In other words, the rest of the quantum circuit might include something like

do $U \in \mathbb{C}^{4 \times 4}$ to qubits 2 and 3 if $b = 1$, else do nothing.

Given that the post-measurement qubit's state is precisely $|b\rangle$, one can instead think of the above conditional-instruction as a “controlled- U gate applied to qubits 1 (control), 2 and 3 (targets),” rather than as some interactive intervention wherein U is applied or not applied, depending on the readout b .



So suppose we're in this case, where we don't really care to know b , we're simply doing some “controlled- U ” quantum gates based off the outcome. And suppose we apply the Deferred Measurement trick from part (a). Since we don't actually care to know the classical bit b , do we really have to do the measurement of the $(n + 1)$ st qubit at the end? (I.e., will the circuit work just as well if we ignore that qubit?) If yes, give an example illustrating the necessity. If not, answer this: do we really have to do the CNOT, either?

2. [The CCNOT Gate.] In class we showed how to simulate classical AND, OR, NOT, and FANOUT gates using only CSWAP gates (and ancillas that could be $|0\rangle$ or $|1\rangle$). Show how to do the same using only CCNOT gates (controlled-CNOT gates, aka Toffoli gates).

3. [Controlled-Controlled- U .] Let U be a d -dimensional qudit gate (i.e., a unitary $d \times d$ matrix). Define the *controlled-controlled- U* gate, which operates on 2 qubits and a qudit (call these the Control1 qubit, the Control2 qubit, and the Target qudit) as follows:

$$\begin{aligned} |00\rangle \otimes |y\rangle &\mapsto |00\rangle \otimes |y\rangle, \\ |01\rangle \otimes |y\rangle &\mapsto |01\rangle \otimes |y\rangle, \\ |10\rangle \otimes |y\rangle &\mapsto |10\rangle \otimes |y\rangle, \\ |11\rangle \otimes |y\rangle &\mapsto |11\rangle \otimes (U|y\rangle). \end{aligned}$$

1. How many rows/columns does this controlled-controlled- U gate have? (I.e., what is the dimension of the space on which it operates?)
2. Prove that the gate is unitary.

3. Suppose V is a (unitary) qudit gate with $V^2 = U$. Show that controlled-controlled- U can be implemented as follows:

- Controlled- V on Control2 and Target.
- CNOT, with Control1 the control and Control2 the target.
- Controlled- $V^\dagger$ on Control2 and Target.
- CNOT, with Control1 the control and Control2 the target.
- Controlled- V on Control1 and Target.

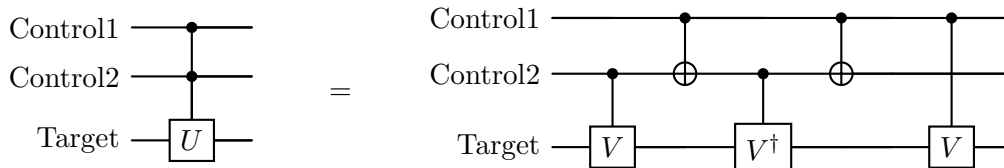


Figure 1: Building controlled-controlled- U

Remark: Earlier we saw the 1-qubit $\sqrt{\text{NOT}}$ gate. If you take this to be V in the above construction, you see that CCNOT can be built out of 2-qubit gates. Interestingly, the classical CCNOT gate *cannot* be computed by any circuit of classical 2-bit gates!

4. [**Deutsch’s Algorithm.**] David really enjoys the fact that one can easily take a classical circuit computing a Boolean function F , and convert it into a quantum circuit which implements the same Boolean function when given “classical inputs” — but which also can accept quantum superpositions of classical inputs. For a whole bunch of small Boolean functions, David built a small quantum circuit that implements that function.

Unfortunately, David forgot to label his quantum circuits, and now he forgets which one computes what! David runs across an old circuit $Q^\pm$ he built which evidently “sign-implements” some 1-bit Boolean function $F : \{0, 1\} \rightarrow \{0, 1\}$. That is, for each $x \in \{0, 1\}$ it holds that

$$Q^\pm |x\rangle = (-1)^{F(x)} |x\rangle$$

but David doesn’t know what F is. Of course there are only four possibilities: $F(x) \equiv 0$, $F(x) \equiv 1$, $F(x) = x$, and $F(x) = \text{NOT}(x)$. Let’s call the first two possibilities “constant functions” and the second two possibilities “nonconstant functions”.

1. Show that it is possible for David to tell whether F is a constant function or a nonconstant function by just using $Q^\pm$ once. Specifically, you should describe a 1-qubit circuit, which may have various gates but only one instance of $Q^\pm$ in it. Your circuit should start with a qubit initialized to $|0\rangle$, and should end with a measurement gate. And it should have the property that based on the measurement outcome, David can know with 100% certainty whether F is a constant function or a non-constant function.

(Hint: Use the good old Rotate, Compute, Rotate paradigm.)

2. Show that if you have access to a classical circuit C computing one of the four possible F ’s, and you only run one bit through it, you cannot gain any information at all about whether F is a constant function or a non-constant function.

2 Week 10.

5. [Classical Query Complexity.] Pythia is holding a strange contest. She is selling a sealed (classical) circuit C that computes some Boolean function $F : \{0, 1\}^n \rightarrow \{0, 1\}$. She promises that $F \not\equiv 0$; that is, there is at least one string x such that $F(x) = 1$. You can buy as many copies of C as you want, but each copy costs a drachma.

Pythia promises you fame and fortune if you can build a (classical) circuit S that outputs a string x such that $F(x) = 1$. The inputs to your circuit S are simply ancillas. You should think of each copy of C that you buy as a big gate that can be used in your circuit S . You are not allowed to “peer into the inner workings of C ”; you can only incorporate it into your circuit as a “black box”.

1. Prove that you can attain fame and fortune by spending $2^n - 1$ drachmas. Actually, it would be too annoying for me to make you describe a circuit, so instead describe classical deterministic pseudocode that incorporates calls to a “subroutine” C . Your code should take no input, make at most $2^n - 1$ calls to C , and end by outputting an x such that $F(x) = 1$.
2. Prove rigorously that it is impossible for you to attain fame and fortune without spending at least $2^n - 1$ drachmas. To be very precise, show that any circuit S incorporating fewer than $2^n - 1$ copies of C cannot be correct, in the sense that there exists an F for which S fails. (No wishy-washiness in your solution; it should be an airtight proof.)
3. The famous “SAT” problem in theoretical computer science is equivalent to the following: Given as input the (description of a) n -input Boolean circuit C , with the promise that there is at least one string $x^* \in \{0, 1\}^n$ such that $C(x^*) = 1$, output any string x such that $C(x) = 1$. If you were able to show that there is no classical algorithm solving this problem in fewer than $2^n - 1$ steps, then you would have shown $P \neq NP$, and real-world fame and fortune would truly be yours. How come your solution to part (b) does not achieve this?

6. [Linear Algebra Modulo 2.] The integers modulo 2 constitute what’s called a “field” in mathematics: a set of numbers (namely 0 and 1) for which all the standard operations of plus, minus, times, and division-by-nonzero work as expected. This “field” is sometimes denoted $\mathbb{F}_2$. (Other examples of fields include the real numbers, the complex numbers, the rational numbers, and the integers modulo p whenever p is prime.) It’s a wonderful fact that pretty much all of linear algebra works just as well when the underlying scalars come from any fixed field, like $\mathbb{F}_2$. The set of all n -dimensional vectors in this case is denoted $\mathbb{F}_2^n$. (You’re most used to the cases $\mathbb{R}^n$ and $\mathbb{C}^n$, when the scalars are reals and complexes, respectively.)

1. There’s pretty much only one pitfall to watch out for: Show that the “dot product” in $\mathbb{F}_2^n$, namely the operation

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i$$

(mod 2 of course), doesn’t act like an “inner product,” in the sense that it’s perfectly possible to have $\mathbf{u} \cdot \mathbf{u} = 0$ even though $\mathbf{u} \neq \mathbf{0}$. Because of this, notions like “orthogonal basis” or “ $\mathbf{u}$ and $\mathbf{v}$ are perpendicular” don’t make as much sense in $\mathbb{F}_2^n$. (We still frequently use the dot product operation anyway, though.)

2. Recall that a set of vectors $\mathbf{u}_1, \dots, \mathbf{u}_k$ is said to be *linearly independent* if the only linear combination

$$c_1 \mathbf{u}_1 + \dots + c_k \mathbf{u}_k$$

that equals $\mathbf{0}$ is the trivial one with $c_1 = c_2 = \dots = c_k = 0$. As usual, the *span* (set of all linear combinations) of a set of k linearly independent vectors is called a k -dimensional subspace. Show that in $\mathbb{F}_2^n$, every k -dimensional subspace contains exactly 2^k vectors.

3. As usual in linear algebra, one can study solutions of systems of linear equations like $Ax = 0$ (where $A \in \mathbb{F}_2^{m \times n}$ is a matrix and x is a vector of n unknowns) or, more generally, $Ax = b$ (where $b \in \mathbb{F}_2^m$ is a fixed right-hand side). The usual facts about Gaussian Elimination apply. Show that the set of solutions x to $Ax = 0$ forms a subspace of dimension equal to $n - r$, where r is the maximum size of a linearly independent set of rows of A .
4. For a more general system $Ax = b$, prove that either there is *no* solution, or else there are 2^{n-r} solutions, where again r is the maximum size of a linearly independent set of rows of A .

7. [Real Quantum Computing.] So far in the course complex amplitudes have almost never arisen, even though they *may*, according to the laws of quantum mechanics. Almost all of our favorite quantum gates (NOT, H , Z , “Rotations”, CNOT, SWAP, CSWAP, CCNOT) are represented by unitary matrices with real entries, meaning they never create states with complex (non-real) amplitudes. We briefly mentioned the 1-qubit phase gate P , whose matrix is

$$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix};$$

i.e., it sends $|0\rangle$ to $|0\rangle$ and $|1\rangle$ to $i|1\rangle$. It’s a fact that if you want the ability to generate all complex unitary transformations, it’s sufficient to: (i) be able to generate all real unitary transformations; and, (ii) be able to apply the P gate.

For this problem, prove that “real amplitudes are sufficient for universal quantum computation”. Specifically, show how to take any quantum circuit that has T gates from the set {real unitaries, measurements, P } and “simply” convert it to an equivalent quantum circuit with no P gates (and hence where all internal states have only real amplitudes). Your new circuit should have just one extra qubit, and it should still have T gates, though some of these gates might act on more qubits than they previously did. (Hint: $(a + bi)|x\rangle$ versus $a|x\rangle|0\rangle + b|x\rangle|1\rangle$.)

8. [The Probability of Coprimality.] Suppose that A and B are chosen independently and uniformly at random from the set $\{1, 2, \dots, S\}$.

1. For P a fixed prime, show that

$$\Pr[A, B \text{ both divisible by } P] \leq \frac{1}{P^2}.$$

2. Show that

$$\Pr[\gcd(A, B) \neq 1] \leq \sum_{\text{primes } P} \frac{1}{P^2}.$$

3. Show that

$$\sum_{\text{primes } P} \frac{1}{P^2} \leq .99,$$

thereby concluding that “The probability that two random integers are coprime is at least 1%.”

Hint. In fact, you don't need calculus; e.g., there's an elementary proof that

$$\sum_{\text{primes } P \geq 5} \frac{1}{P^2} \leq \frac{2}{5},$$

and hence

$$\sum_{\text{primes } P} \frac{1}{P^2} \leq .77 \dots$$

9. [A simple Boolean Fourier formula.]

Let $f : \{0, 1\}^n \rightarrow \mathbb{C}$. In class we saw the following nice fact:

$$s = 000 \dots 0 \implies \hat{f}(s) = \mathbb{E}_{x \sim \{0,1\}^n} [f(x)],$$

where $\mathbb{E}_{x \sim \{0,1\}^n} [\cdot]$ denotes the expected value when x is chosen uniformly at random from $\{0, 1\}^n$.

Prove also the following formula:

$$s \neq 000 \dots 0 \implies \hat{f}(s) = \frac{1}{2} (\mathbb{E}_{x \sim \{0,1\}^n} [f(x) \mid \chi_s(x) = +1] - \mathbb{E}_{x \sim \{0,1\}^n} [f(x) \mid \chi_s(x) = -1]),$$

where the $\mid$ notation denotes conditional expectation.

10. [Hands-on XOR-pattern practice.]

1. Let $\text{AND} : \{0, 1\}^2 \rightarrow \{0, 1\}$ be the logical-AND function on two bits.

- (a) Write the full truth-table of AND.
- (b) Let $\text{and} : \{0, 1\}^2 \rightarrow \{\pm 1\}$ be defined by

$$\text{and}(x) = (-1)^{\text{AND}(x)}.$$

Write the full table of function values for and.

- (c) Write the quantum state $|\text{and}\rangle$ in standard bra-ket notation.
- (d) For this problem, if $g : \{0, 1\}^n \rightarrow \mathbb{C}$ is a function, let $[g]$ denote the column vector in $\mathbb{C}^N$ of g 's values (where $N = 2^n$).

Write the four length-4 column vectors $[\chi_s]$, where $\chi_s : \{0, 1\}^2 \rightarrow \{\pm 1\}$ are the XOR functions corresponding to the 2-bit Boolean Fourier transform.

- (e) Compute $\widehat{\text{and}}(s)$ for each $s \in \{0, 1\}^2$.
- (f) Using your solutions above, write the explicit vector equation

$$[\text{and}] = \widehat{\text{and}}(00)[\chi_{00}] + \widehat{\text{and}}(01)[\chi_{01}] + \widehat{\text{and}}(10)[\chi_{10}] + \widehat{\text{and}}(11)[\chi_{11}],$$

then write “Yep.”

2. Repeat parts (ii), (v), (vi) for the function $\text{MAJ} : \{0, 1\}^3 \rightarrow \{0, 1\}$ defined by

$$\text{MAJ}(x_1, x_2, x_3) = \text{the majority bit among } x_1, x_2, x_3.$$

(Hint: you might want to use the result in Problem 9.)

3. Repeat parts (ii), (v), (vi) for the function

$$\text{SORT} : \{0, 1\}^4 \rightarrow \{0, 1\}$$

defined by

$$\text{SORT}(x_1, x_2, x_3, x_4) = 1$$

iff

$$x_1 \leq x_2 \leq x_3 \leq x_4 \quad \text{or} \quad x_1 \geq x_2 \geq x_3 \geq x_4.$$

(Honestly, you might want to get a computer to help you with this.)

11. [Deutsch–Jozsa.]

David and Richard enjoy the fact that one can easily take a classical circuit computing a Boolean function F and convert it into a quantum circuit that implements the same Boolean function but can also accept quantum superpositions of classical inputs.

They did this for several functions, including

- the constantly-0 function $F : \{0, 1\}^n \rightarrow \{0, 1\}$,
- various balanced functions, meaning $F(x) = 0$ for half the inputs and $F(x) = 1$ for half.

They find a circuit $Q^\pm$ which sign-implements some function $F : \{0, 1\}^n \rightarrow \{0, 1\}$ but they do not know whether F is all-0 or balanced.

1. Show that David and Richard can determine whether F is all-0 or balanced using $Q^\pm$ once.

(Hint: Fourier sampling.)

2. Suppose instead you only have access to a classical circuit C computing F , promised to be either all-0 or balanced.

Show that if you act deterministically you cannot distinguish the two cases unless you evaluate C on more than 2^{n-1} inputs.

3. Suppose you may use randomness.

Show that by evaluating C on only T inputs you can distinguish the two cases with one-sided error 2^{-T} .

12. [Translated Fourier coefficients.]

Let $f : \{0, 1\}^n \rightarrow \mathbb{C}$. For $y \in \{0, 1\}^n$ define

$$f^{+y}(x) = f(x + y),$$

where addition is over $\mathbb{F}_2^n$. Compute $\widehat{f^{+y}}(s)$ in terms of $\hat{f}(s)$. How does performing Fourier sampling on f^{+y} compare to Fourier sampling on f ?

13. [Complex roots of unity.]

1. Review, if necessary, Problem 2 on Weekly Work 2.
2. Let M be a positive integer and let $\omega_M \in \mathbb{C}$ be the primitive M -th root of unity. Let $0 \leq t < M$. Compute

$$\text{avg}_{u \in \{0,1,2,\dots,M-1\}} \{\omega_M^{tu}\}.$$

There should be two possible outcomes depending on t .

14. [Subspaces and Fourier transforms.]

Recall the vector space $\mathbb{F}_2^n$.

1. Suppose $A \subseteq \mathbb{F}_2^n$ is a linear subspace of dimension k . Let

$$A^\perp = \{s \in \mathbb{F}_2^n : s \cdot x = 0 \ \forall x \in A\}.$$

Show that $A^\perp$ is a subspace of dimension $n - k$.

2. Give an example with $n = 2$ of a subspace A of dimension 1 such that $A^\perp = A$.
3. Show that $(A^\perp)^\perp = A$.
4. Given a subspace A of dimension k , define

$$g(x) = \begin{cases} \sqrt{\frac{N}{2^k}} & x \in A \\ 0 & x \notin A \end{cases}$$

where $N = 2^n$.

Compute

$$H^{\otimes n}|g\rangle,$$

equivalently compute $\hat{g}(s)$ for each $s \in \{0,1\}^n$.

3 Week 11.

15. [The cyclic group.]

The cyclic group of size N is the set $\mathbb{Z}_N$ of integers modulo N , together with the operation of addition modulo N . The neutral element is 0, and the inverse of $u \in \mathbb{Z}_N$ is $-u = N - u$.

1. The order of an element $u \in \mathbb{Z}_N$, denoted $\text{ord}_{\mathbb{Z}_N}(u)$ or just $\text{ord}(u)$, is the least positive integer r such that

$$u + u + \cdots + u = 0$$

(r times).

Print a table showing all elements of $\mathbb{Z}_{96}$ together with their orders. You will want to use a computer.

2. Prove that

$$\text{ord}(u) = \frac{N}{\gcd(u, N)} = \frac{\text{lcm}(u, N)}{u},$$

where lcm denotes least common multiple (with u, N treated as natural numbers). Ignore the lcm formula when $u = 0$.

3. Let 2^K be the largest power of 2 dividing N , and assume $K \geq 1$.

Prove:

- If $u \in \mathbb{Z}_N$ is odd, then the largest power of 2 dividing $\text{ord}(u)$ is 2^K .
- If $u \in \mathbb{Z}_N$ is even, then the largest power of 2 dividing $\text{ord}(u)$ is not 2^K .

4. Let N_1, N_2 be even integers. Pick $u_1 \in \mathbb{Z}_{N_1}$ and $u_2 \in \mathbb{Z}_{N_2}$ independently and uniformly at random.

Prove that with probability at least $1/2$ the largest power of 2 dividing $\text{ord}_{\mathbb{Z}_{N_1}}(u_1)$ differs from the largest power of 2 dividing $\text{ord}_{\mathbb{Z}_{N_2}}(u_2)$.

5. Let N_1, N_2 be positive integers.

Define the group $\mathbb{Z}_{N_1} \times \mathbb{Z}_{N_2}$ with operation

$$(u_1, u_2) + (v_1, v_2) = (u_1 + v_1 \bmod N_1, u_2 + v_2 \bmod N_2).$$

The neutral element is $(0, 0)$ and the inverse of (u_1, u_2) is $(-u_1, -u_2)$.

For $u = (u_1, u_2)$ define the order $\text{ord}_{\mathbb{Z}_{N_1} \times \mathbb{Z}_{N_2}}(u)$ to be the least positive r satisfying

$$u + u + \cdots + u = 0.$$

Prove

$$\text{ord}_{\mathbb{Z}_{N_1} \times \mathbb{Z}_{N_2}}(u) = \text{lcm}(\text{ord}_{\mathbb{Z}_{N_1}}(u_1), \text{ord}_{\mathbb{Z}_{N_2}}(u_2)).$$

16. [The last obligatory number theory problem?]

Recall that in Shor's factoring algorithm the input was $B = P \cdot Q$, the product of two odd primes. We attempted to find a nontrivial square root R of 1 (mod B).

Strategy:

- Pick $A \in \mathbb{Z}_B^*$ uniformly at random.
- Use a quantum computer to determine the order L of A , the least positive integer such that

$$A^L \equiv 1 \pmod{B}.$$

- Hope that
 - L is even.
 - $A^{L/2} \not\equiv -1 \pmod{B}$.

If both occur, set

$$R = A^{L/2} \pmod{B}.$$

Using the isomorphisms

$$\mathbb{Z}_B^* \cong \mathbb{Z}_P^* \times \mathbb{Z}_Q^* \cong \mathbb{Z}_{P-1} \times \mathbb{Z}_{Q-1},$$

we obtain the equivalent experiment:

- Choose $u_1 \in \mathbb{Z}_{P-1}$ and $u_2 \in \mathbb{Z}_{Q-1}$ uniformly.
- Let $u = (u_1, u_2)$.
- Let $L = \text{ord}_{\mathbb{Z}_{P-1} \times \mathbb{Z}_{Q-1}}(u)$.
- Hope that
 - L is even.
 - $u + \dots + u$ ($L/2$ times) is not $(\frac{P-1}{2}, \frac{Q-1}{2})$.

Prove that if the largest power of 2 dividing $\text{ord}_{\mathbb{Z}_{P-1}}(u_1)$ differs from the largest power of 2 dividing $\text{ord}_{\mathbb{Z}_{Q-1}}(u_2)$ then both lucky conditions occur.

Conclude that the probability of success is at least $1/2$.

17. [Order-finding reduces to factoring.]

Suppose you had a subroutine that could factor any number $B = P \cdot Q$.

Using this subroutine, give an efficient deterministic algorithm for finding the order of $A \in \mathbb{Z}_B^*$ given A and B .

Conclude that factoring and order-finding have equivalent complexity.

18. [The semiclassical Quantum Fourier Transform.]

This problem concerns the semiclassical implementation of the QFT.

1. Recall the controlled phase gates used in the QFT.

If the control qubit is $|1\rangle$ then the unitary V_j is applied to the target qubit, where V_j leaves $|0\rangle$ unchanged and applies phase

$$\omega_N^{2^{n-1-j}}$$

to $|1\rangle$.

Show these two-qubit gates are symmetric: the same operation results if the control and target are reversed.

2. Using this fact, show that the usual QFT circuit can be redrawn with all controlled phase gates symmetric.

Assume we plan to measure all qubits immediately afterwards.

Show that we obtain equivalent results if we measure each qubit immediately after its Hadamard gate and use the classical outcome to decide whether to apply the single-qubit V_j gates.

4 Week 12.

19. [Cosets.]

Let G be a finite group with operation $\circ$. Let H be a subgroup of G .

1. Show that for every $x \in G$, the coset xH has the same number of elements as H .
2. Show that any two cosets xH and yH are either identical or disjoint.
3. Show that the set of cosets of H forms a partition of G .

20. [Graph Isomorphism via the Hidden Subgroup Problem.]

Let G be a graph on vertices $\{1, \dots, n\}$ with edge set E . Let S_n denote the group of permutations.

Define $\text{Aut}(G)$ as the permutations $\pi \in S_n$ satisfying

$$E = \{\{\pi(u), \pi(v)\} : u, v \in E\}.$$

1. Show that if $\pi, \sigma \in \text{Aut}(G)$ then $\pi \circ \sigma$ and π^{-1} are also in $\text{Aut}(G)$.
Conclude $\text{Aut}(G)$ is a subgroup of S_n .
2. Let COLORS be the set of adjacency matrices of n -vertex graphs.
Define $f_G : S_n \rightarrow \text{COLORS}$ where $f_G(\pi)$ is the adjacency matrix obtained by permuting G by π .
Show f_G is $\text{Aut}(G)$ -periodic.
3. Show there is a polynomial-time classical algorithm that, given G , outputs a classical circuit computing f_G .
4. Let G_1, G_2 be connected graphs. Let $G = G_1 \sqcup G_2$.
Show G_1 and G_2 are isomorphic iff there exists $\sigma \in \text{Aut}(G)$ mapping a vertex $i \leq n$ to some $j > n$.
5. Suppose generators $h_1, \dots, h_k$ for a subgroup $H \subseteq S_{2n}$ are given.
Show one can efficiently determine whether some $\sigma \in H$ maps between halves.
6. Suppose there were an efficient quantum algorithm for the Hidden Subgroup Problem in S_{2n} .
Show this would yield an efficient quantum algorithm for Graph Isomorphism.